Extract From PostgreSQL Database

Explore how to extract data from PostgreSQL databases using Python's psycopg2 library. Learn to connect, query, and export data, and understand how to leverage PLpgSQL to add complex logic like conditional salary updates within your extraction process.

Introducing PostgreSQL

Another common relational database used by the industry is PostgreSQL. PostgreSQL is open-sourced and has been around since 1996 so it's a stable database with great support and community. It's used primarily as a database for web and analytics applications.

While MySQL is a simple and fast OLTP database, PostgreSQL is a feature-rich database that can handle complex analytical queries and can be used for both OLTP and OLAP use cases.

One of the main advantages of PostgreSQL over MySQL is that PostgreSQL offers support for multiple programming languages, most notably **PL/pgSQL**.

PL/pgSQL is a procedural programming language for the PostgreSQL database system. It allows users to extend the functionality of PostgreSQL by adding complex logic, for example, control structures such as if, case, for, and while loop statements and more.

Many large organizations use PostgreSQL. Some examples are:

Ask

- Uber

- Netflix
- Instagram
- Spotify
- Instacart
- LaunchDarkly
- Robinhood
- Reddit

Extracting data from PostgreSQL

We can easily extract data from a PostgreSQL database using Python's library called `psycopg2`.

`psycopg2` is a PostgreSQL driver that allows us to connect to and query a PostgreSQL database using Python.

First, we need to acquire the connection details of an active PostgreSQL database, the connection details are:

- `host`
- `port`
- `user`
- `password`
- `database`

Next, we use Python to perform queries on the database. For example, let's extract the top two highest-paid employees in the company and export the data as CSV. The employees' details are stored in the following table:

Ask

main.sql PostgreSQL

```
CREATE TABLE employees (
```

```
  id serial PRIMARY KEY,
  first_name varchar(50) NOT NULL,
  last_name varchar(50) NOT NULL,
  job_title varchar(100) NOT NULL,
  hire_date date NOT NULL,
  salary numeric NOT NULL
);

INSERT INTO employees (first_name, last_name, job_title, hire_date, salar
VALUES
  ('John', 'Doe', 'Software Engineer', '2022-01-01', 90000),
```

Creating an example table in PostgreSQL

Now, let's extract and import the data using Python.

main.py Python 3.10.4

```
import psycopg2

# Connect to the database
conn = psycopg2.connect(host='localhost', port = 5432, database='postgres

# Create a cursor
cur = conn.cursor()

# Execute a SQL query
cur.execute('''
SELECT
  first_name,
  last_name
```

Run

Querying the top two highest-paid employees

> **Note:** Any extraction methods used on MySQL can also be applied to PostgreSQL, such as full or incremental extraction or extracting data from a log file.

Ask

As mentioned, we can also include complex logic in our queries using PL/pgSQL. For example, the following script uses a for loop to traverse each employee in the table and grant a salary raise for employees with a tenure of over one year and a salary under 100,000.

For each employee, we calculate the employee's tenure and current salary and if they meet the conditions, the following script will raise their salary by 5% and print a notification.

main.py Python 3.10.4

```
import psycopg2

# Connect to the database
conn = psycopg2.connect(host='localhost', port = 5432, database='postgres

# Create a cursor
cur = conn.cursor()

# Execute a SQL query
cur.execute('''
do $$
declare
    rec record;
```

Run

Raising employee salaries by 5% using PL/pgSQL

- **Line 11:** We initiate a PL/pgSQL script. We set the escape character for the script to be `$$` instead of `;`.
- **Lines 12–15:** We declare three variables. `rec` will store each row in the for loop, `employee_tenure` will store each employee's tenure, and `employee_salary` will store each employee's salary.
- **Line 16:** We initiate a transaction.
- **Line 19:** We loop over each employee using a `for` loop.

Ask

- **Lines 22–25:** We calculate an employee's tenure by subtracting the current year from the hiring year.
- **Lines 28–31:** We extract each employee's salary.
- **Lines 34–37:** We grant a 5% salary raise to employees who meet the requirements.
- **Line 40:** We raise a notification regarding the employee who received a salary raise.

Ask